

Secure Code Review and Vulnerability Assessment of a Python Login System

Introduction

Software applications often contain security vulnerabilities that attackers can exploit to gain unauthorized access or steal sensitive information. This project focuses on performing a security audit of a simple Python-based login application. The objective is to identify vulnerabilities, analyze their impact, and recommend secure coding practices.

Application Selected

Programming Language: Python

Application: User Login System

The application allows users to enter a username and password for authentication. A review was conducted to identify potential security weaknesses.

Vulnerable Code

```
username = input("Enter username: ")  
password = input("Enter password: ")
```

```
query = "SELECT * FROM users WHERE username='" + username + "' AND  
password='" + password + "'"
```

```
cursor.execute(query)
```

Security Vulnerabilities Identified

1. SQL Injection

Description:

The application directly concatenates user input into SQL queries. Attackers can manipulate the query and bypass authentication.

Severity: High

Impact: Unauthorized access to the application and database.

2. Plain Text Password Storage

Description:

Passwords are stored and processed in plain text format.

Severity: High

Impact: Passwords can be exposed if the database is compromised.

3. Lack of Input Validation

Description:

The application accepts user input without validation or sanitization.

Severity: Medium

Impact: Increased risk of injection attacks and unexpected application behavior.

Security Analysis Tool Used

Bandit

Bandit is a static analysis tool designed to find security issues in Python code.

Installation Command:

```
pip install bandit
```

Execution Command:

```
bandit login.py
```

Bandit helps identify insecure coding practices and potential vulnerabilities.

Secure Code Implementation

```
import sqlite3
```

```
username = input("Enter username: ")
```

```
password = input("Enter password: ")
```

```
conn = sqlite3.connect("users.db")
```

```
cursor = conn.cursor()
```

```
cursor.execute(
```

```
    "SELECT * FROM users WHERE username=? AND password=?",
```

```
(username, password)
)
```

Recommendations

1. Use parameterized SQL queries.
2. Store passwords using secure hashing algorithms such as bcrypt.
3. Validate and sanitize all user inputs.
4. Implement proper error handling.
5. Follow OWASP secure coding guidelines.
6. Regularly update software dependencies.
7. Conduct periodic security audits.

Findings Summary

Vulnerability	Severity	Recommendation
SQL Injection	High	Parameterized Queries
Plain Text Passwords	High	Password Hashing
No Input Validation	Medium	Input Validation

Conclusion

The security audit identified several vulnerabilities in the Python login application. SQL Injection and insecure password handling were the most critical issues. Implementing secure coding practices, using static analysis tools, and following industry security standards can significantly improve application security and reduce the risk of cyberattacks.